R Functions Lab

Background
In this session you will work through the process of developing your own functions for generating random DNA and protein sequences.

The process will involve starting slowly with small defined segments of the code. Then building up to string them together into simple functions.

We will walk through the first two functions (add and generate_dna) together in class and then you will get to work on your own on the last one (generate_protein). As always, if you have questions, please ask in person or on Piazza. These are important skills and we want to support you in attaining them as best we can!

Your Tasks

- Create a new RStudio project for this class session. Note that you should save this inside the folder where you have been organizing your work to date for this class.
- Create a new Quarto document (or Rmarkdown document if you do not have Quarto available) for storing your work (i.e. notes and code for this session). You will Render this to a pdf lab report for later submission on gradescope.
- Answer each of the following questions.

Q1: Write a function *add()* to add some numbers together. Make sure to add comment(s) to the function explaining why you do things.

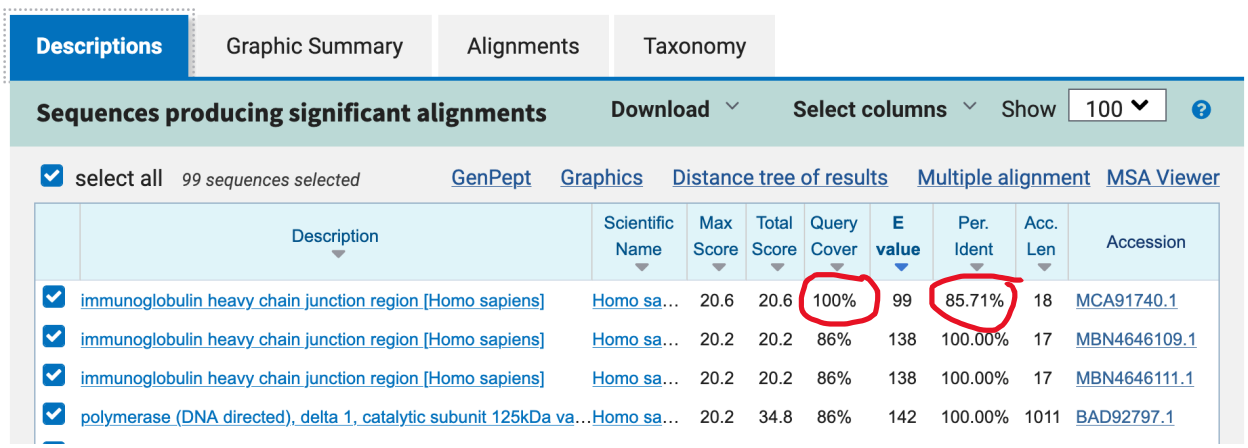
a. Your first version of the function should add two input numbers together; e.g. add(4, 7) should return 11. [**1 pt**]
b. Your second version of the function should add any numbers together that the user inputs; e.g. add(1, 2, 3, -4) should return 2. [**1 pt**]

Q2: Write a function *generate_dna()* that generates a random DNA sequence of a length input by the user; e.g. *generate_dna(11)* may return "ATTTAAGGCTG". Make sure to add comments to the function. Play with functions *sample()* and *paste()* in your R-Studio Console to see how they can help here. [**1 pt**]

Q3: Write a function *generate_protein()* that generates a random peptide/protein sequence of a length input by the user; e.g. *generate_protein(6)* may return "WQRTAG". Make sure to use the single-letter code for all 20 amino acids and use no other letters (see list of amino acids at the bottom of the page); make sure that individual amino acids <u>can appear more than once</u>. Make sure to add comments to the function. [**1 pt**]

Q4: Use your *generate_protein()* function to generate peptides of lengths 5, 7, 9, and 11 amino acids. Take advantage of the base R function **sapply()** for this. [**1 pt**]

Q5: Take each of your peptides from Q4 and run a BLASTP search (https://blast.ncbi.nlm.nih.gov/Blast.cgi) selecting the ***Non-redundant protein sequences (nr)*** database and ***homo sapiens (taxid: 9606)*** organism. For each search, before clicking BLAST check the ***Show results in a new window*** tab next to the BLAST button to easily allow you to run the blast search for all four peptides at the same time in four different tabs. Once each search is done, for the top aligned protein for each peptide, list the percent identity (Per. Ident) *times* coverage (Query Cover) (for example, in the example below, 100% Query Cover *times* 85.71% Per. Ident = 100% x 85.71% = 1.00 x 0.8571 x 100% = 85.71%); make sure to type out your calculations in your Quarto document [note: if any of your values are below 30% you are probably miscalculating something]. Upload your numbers for each peptide (length (aa) and max identity (%)) into the shared class excel data sheet (see *Data Sheet* link in the Canvas assignment). Also, for each datapoint, add your initials in the "Your_Initials" column of the data sheet. Before adding your initials, scan through other used initials; if anyone has the same initials as you, modify your initials to make them unique (your initials will be used below to identify your specific data points in a ggplot graph). [**1 pt**]



Description	Scientific Name	Max Score	Total Score	Query Cover	E value	Per. Ident	Acc. Len	Accession
immunoglobulin heavy chain junction region [Homo sapiens]	Homo sa...	20.6	20.6	100%	99	85.71%	18	MCA91740.1
immunoglobulin heavy chain junction region [Homo sapiens]	Homo sa...	20.2	20.2	86%	138	100.00%	17	MBN4646109.1
immunoglobulin heavy chain junction region [Homo sapiens]	Homo sa...	20.2	20.2	86%	138	100.00%	17	MBN4646111.1
polymerase (DNA directed), delta 1, catalytic subunit 125kDa va...	Homo sa...	20.2	34.8	86%	142	100.00%	1011	BAD92797.1

--- Take a break to allow a good amount of data to be uploaded into the shared excel sheet ---

Q6: Once there is a good amount of data in the shared excel sheet, export the sheet as a CSV UTF-8 file (*Export* option under the *File* tab) and save the file to your class working folder (i.e. the folder in which you are currently generating a Quarto document). In R-Studio, upload the csv file into a data frame using the command **peptide_df <- read.csv("your_filename")**. Use the **head()** function to list the first **10 rows** (note that the default for *head()* is 6) of the *peptide_df* dataframe as a test that everything looks good. [**1 pt**]

Q7: Use ggplot2 to generate a box plot with Length_aa on the x-axis and Max_Identity_perc on the y-axis. Since the box plot will require categorical values for Length_aa, you may have to specify the x-axis as *as.factor(Length_aa)*. Feel free to pretty up the plot by adding your own x/y-axis labels, graph title, etc using the *labs()* function and/or using your favorite theme using a *theme()* function (see the *data-visualization_ggplot.pdf* cheat sheet in Canvas for options). Optional: you will reuse this plot a few times below, so your subsequent coding will be simpler if you assign your box plot to an object such as 'p' (p <- ggplot(peptide_df) +....etc). [**1 pt**]

Q8: Let's add a *geom_jitter()* layer to your box plot to visualize the individual data points on top of the box plot and highlight your own data points.

a. First, add a *geom_jitter()* layer to your box plot with each data point colored according to students' initials by adding the following additional line to your plot: + **geom_jitter(aes(col=Your_Initials))**. Feel free to narrow the spread of the jitter points using the width argument (e.g. width = 0.2). You can find your own datapoints this way via the color legend, but it looks rather messy. [**1 pt**]

b. Now let's control the coloring ourselves to better highlight your own data points. We can do this by generating a vector with a color for each datapoint (we will use your favorite color for your datapoints and "GREY" for all other datapoints). You can then use this vector to assign colors with the *geom_jitter()* call.

   i. First, use the *rep()* function to generate a vector consisting of as many repeats of "GREY" as there are datapoints in the *peptide_df* data frame. Assign this vector to an object you name *color*: i.e. **color <- c(rep(__,__))** (fill in the blanks). Once you have generated the *color* object, use the *table()* function to ensure that you have the correct number of instances of "GREY" in *color*. [**1 pt**]

      - Hints for (i): Use your R-Studio Console to test the outputs of *rep("GREY", 2)*, *dim(peptide_df)*, and *dim(peptide_df)[1]*, and make sure to look at documentation for the *dim()* function (?dim) to learn what the output numbers represent.

   ii. Next, use the Your_Initials column in the *peptide_df* dataframe (which can be called as *peptide_df$Your_Initials*) to generate a TRUE/FALSE vector indicating which datapoints are yours (TRUE) and which are not (FALSE). Assign this vector to an object you name *mydata*: i.e. **mydata <- _____** (fill in the blank). Again, check that you have the correct number of TRUE and FALSE in *mydata* using the *table()* function. [**1 pt**]

      - Hint for (ii): Use your R-Studio Console to test the outputs of *c("JLA", "MK", "BG") == "JLA"* and of *peptide_df$Your_Initials*

   iii. Now, use the *mydata* vector to replace the "GREY" values in the color vector that correspond to your data points with your favorite color, like this: **color[mydata] <- "COLOR"** (replace "COLOR" with the name of your favorite color; see link at the bottom of the page to a webpage with 657 R color names to select a cool color). Note, this will replace "GREY" with your favorite color for all the

positions that are TRUE in *mydata*, i.e. the positions of your data points. Again, check the vector with *table()*. [**1 pt**]

iv. Now you are ready to remake the plot by adding the following line to your original box plot from Q7: **+ geom_jitter(col=color)**. (Note that the color call is not run as an aesthetic here because we are now directly telling ggplot what color to use for each dot). Again, feel free to use the width argument to narrow the jitter points if you wish (e.g. width = 0.2). [**1 pt**]

c. Based on your boxplot, does any of your own peptide data fall outside the 25 to 75 percentile of the class's data? If yes, list which one(s). [**1 pt**]

Q9: MHC class II molecules of our immune system display 9 amino acid peptides on the surface of immune cells that, when from a foreign origin, activates a T-cell immune response. Based on your findings in Q7 and 8, speculate why those peptides are 9 amino acids long rather than, say, 5 amino acids. [**1 pt**]

Make sure you save your Quarto document and click the "Render" (or Rmarkdown "Knit") button to generate a PDF format report without errors. Finally, submit your PDF to gradescope; make sure to indicate on which page your answers to individual questions (Q1a, Q1b, Q2, etc) can be found.

List of amino acids:
"A", "R", "N", "D", "C", "E", "Q", "G", "H", "I", "L", "K", "M", "F", "P", "S", "T", "W", "Y", "V"

R Color names:
https://www.datanovia.com/en/blog/awesome-list-of-657-r-color-names